Dot Net Practical

Practical No: 1

1. **Write the program for the following**

a. **Create an MVC application to perform basic Arithmetic Operations.**

Controller Code:

```
using Microsoft.AspNetCore.Mvc;

namespace MvcCalculator.Controllers

{

public class CalculatorController : Controller

{

public IActionResult Index

{

 return View();

}


[HttpPost]

public IActionResult Calculate(double num1, double num2, string operation)

{

 double result = 0;

 switch (operation)

 {
```

```
            case "Add": result = num1 + num2; break;
            case "Subtract": result = num1 - num2; break;
            case "Multiply": result = num1 * num2; break;
            case "Divide":
                result = num2 != 0 ? num1 / num2 : 0;
                break;
        }
        ViewBag.Result = result;
        return View("Index");
    }
  }
}
```

View Code:

```
@{
  ViewData["Title"] = "Calculator";
}

<div style="margin: 30px;">
  <h2>MVC Arithmetic Calculator</h2>
  <form method="post" asp-action="Calculate">
    <label>Number 1:</label><br />
    <input type="number" name="num1" required style="padding: 5px;
margin: 5px 0;" /><br />

    <label>Number 2:</label><br />
```

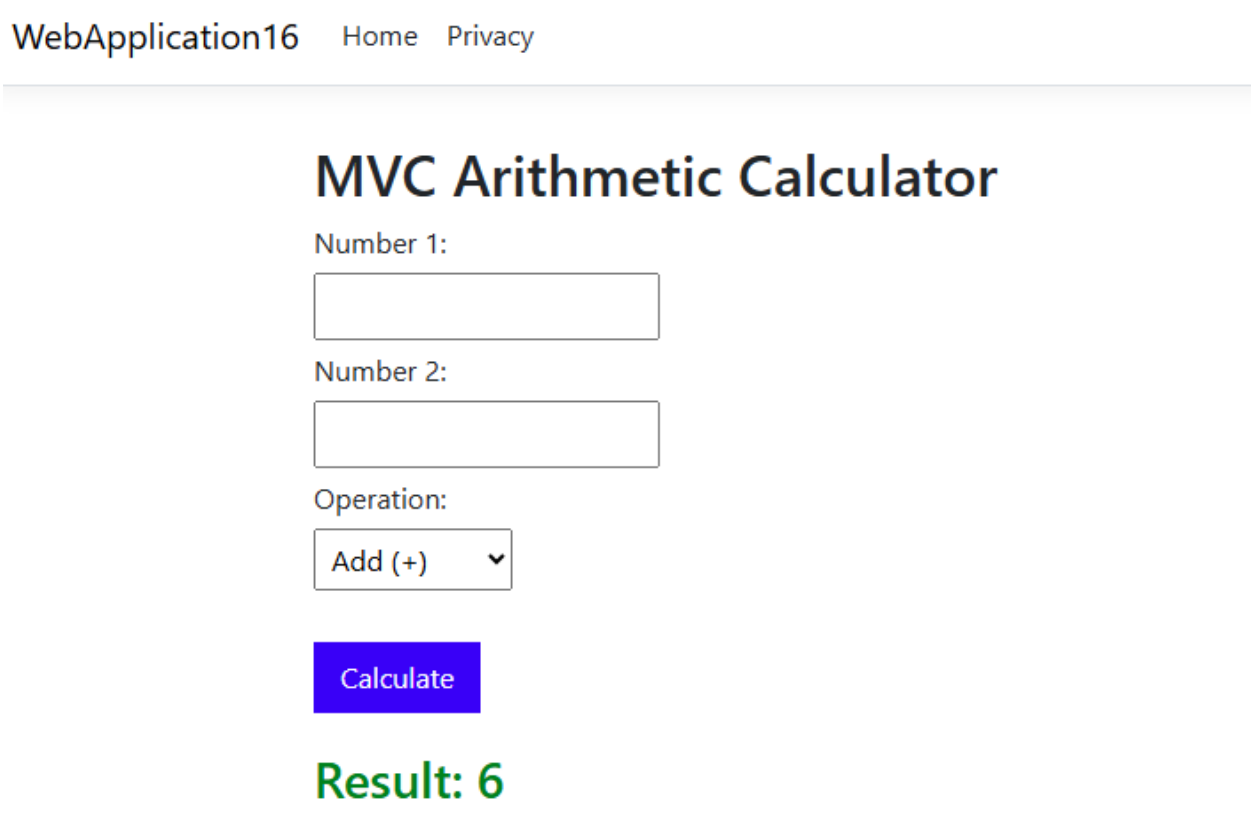
```
    <input type="number" name="num2" required style="padding: 5px; margin: 5px 0;" /><br />

    <label>Operation:</label><br />
    <select name="operation" style="padding: 5px; margin: 5px 0;">
        <option value="Add">Add (+)</option>
        <option value="Subtract">Subtract (-)</option>
        <option value="Multiply">Multiply (*)</option>
        <option value="Divide">Divide (/)</option>
    </select><br /><br />

    <button type="submit" style="padding: 8px 15px; background: blue; color: white; border: none;">Calculate</button>
</form>

@if (ViewBag.Result != null)
{
    <h3 style="color: green; margin-top: 20px;">Result: @ViewBag.Result</h3>
}
</div>
```

Output:



b. Create an application to print Floyd's triangle till n rows in C#.

Code:

```
using System;


class Program
{
    static void Main()
    {
        Console.Write("Enter number of rows for Floyd's Triangle: ");
        int n = int.Parse(Console.ReadLine());
```

```
        int k = 1;


        for (int i = 1; i <= n; i++)
        {
            for (int j = 1; j <= i; j++)
            {
                Console.Write(k + " ");
                k++;
            }
            Console.WriteLine();
        }
    }
}
```

Output:

```
Enter number of rows for Floyd's Triangle: 10
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
16 17 18 19 20 21
22 23 24 25 26 27 28
29 30 31 32 33 34 35 36
37 38 39 40 41 42 43 44 45
46 47 48 49 50 51 52 53 54 55
```

c. Write C# code to arrange the name of cities in sorted order. Accept name of 10 cities from the user

Code:

```
using System;

class Program
```

```
{
    static void Main()
    {
        string[] cities = new string[10];

        Console.WriteLine("Enter names of 10 cities:");

        for (int i = 0; i < 10; i++)
        {
            Console.Write($"City {i + 1}: ");
            cities[i] = Console.ReadLine();
        }
        Array.Sort(cities);

        Console.WriteLine("\nCities in Sorted Order:");

        foreach (string city in cities)
        {
            Console.WriteLine(city);
        }
    }
}
```

Output:

```
Enter names of 10 cities:
City 1: Mumbai
City 2: Pune
City 3: Chennai
City 4: Banglore
City 5: Delhi
City 6: Punjab
City 7: Rajasthan
City 8: Kashmir
City 9: Uttrakand
City 10: Hyderabad

Cities in Sorted Order:
Banglore
Chennai
Delhi
Hyderabad
Kashmir
Mumbai
Pune
Punjab
Rajasthan
Uttrakand
```

Practical No: 2

Write the program with different features of C#

a. **Function Overloading**

Code:

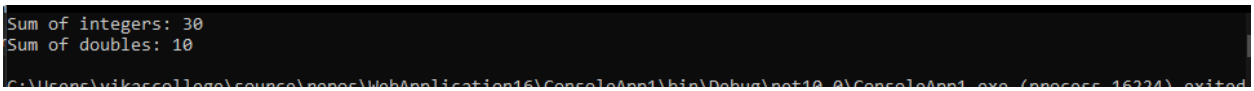
```
using System;

class Calculator
{
   public int Add(int a, int b) { return a + b; }
   public double Add(double a, double b) { return a + b; }

   static void Main()
   {
      Calculator calc = new Calculator();
```

```
        Console.WriteLine("Sum of integers: " + calc.Add(10, 20));
        Console.WriteLine("Sum of doubles: " + calc.Add(5.5, 4.5));
    }
}
```

Output:



b. Inheritance (all types):

Code:

```
using System;

// Single / Base Class
class Animal
{
    public void Eat() { Console.WriteLine("Eating..."); }
}

// Single / Multilevel derived class
class Dog : Animal
{
    public void Bark() { Console.WriteLine("Barking..."); }
}

class Program
{
    static void Main()
    {
        Dog d = new Dog();
```

```
        d.Eat();  // Inherited from Animal
        d.Bark(); // Defined in Dog
    }
}
```

Output:



C. Constructor overloading

Code:

```
using System;

class Student
{
    public string Name;
    public int Age;

    // Default Constructor
    public Student()
    {
        Name = "Unknown";
        Age = 0;
    }

    // Parameterized Constructor
    public Student(string name, int age)
    {
        Name = name;
        Age = age;
    }

    public void Display()
```

```
    {
        Console.WriteLine($"Name: {Name}, Age: {Age}");
    }

    static void Main()
    {
        Student s1 = new Student();
        Student s2 = new Student("Aakash", 21);

        s1.Display();
        s2.Display();
    }
}
```

Output:



d. Interfaces

Code:

```
using System;

interface IVehicle
{
    void Start();
}

class Car : IVehicle
{
    public void Start()
    {
```

```
        Console.WriteLine("Car is starting with key.");
    }
}

class Program
{
    static void Main()
    {
        IVehicle v = new Car();
        v.Start();
    }
}
```

Output:

```
Car is starting with key.
```

e. Using Delegates and events:

Code:

```
using System;

class Program
{
    // 1. Declare Delegate
    public delegate void Notify();

    // 2. Declare Event using Delegate
    public static event Notify OnProcessCompleted;

    static void Main()
    {
        // Subscribe to event
```

```
        OnProcessCompleted += () => Console.WriteLine("Process
Completed Event Triggered!");

        Console.WriteLine("Processing task...");

        // Trigger event
        OnProcessCompleted?.Invoke();
    }
}
```

Output:

```
Processing task...
Process Completed Event Triggered!
```

f. **Exception handling:**

Code:

```
using System;

class Program
{
    static void Main()
    {
        try
        {
            int a = 10, b = 0;
            int c = a / b; // Will cause DivideByZeroException
        }
        catch (DivideByZeroException)
        {
            Console.WriteLine("Error: Cannot divide by zero!");
        }
```

```
        finally
        {
            Console.WriteLine("Finally block executed.");
        }
    }
}
```

Output:



Practical No: 3

Write the program for the following:

a. **Create methods add(). multiply(), substract() ,divide() with suitable parameters and call these methods using concept of C# delegate.**

Code:

```
using System;

namespace Practical3A
{
    // Step 1: Declare a delegate with the same signature as the methods
    public delegate void MathOperation(double a, double b);

    class Program
    {
        // Step 2: Create the methods
        public static void Add(double a, double b)
        {
            Console.WriteLine($"Addition: {a} + {b} = {a + b}");
```

```
    }

    public static void Subtract(double a, double b)
    {
        Console.WriteLine($"Subtraction: {a} - {b} = {a - b}");
    }

    public static void Multiply(double a, double b)
    {
        Console.WriteLine($"Multiplication: {a} * {b} = {a * b}");
    }

    public static void Divide(double a, double b)
    {
        if (b != 0)
            Console.WriteLine($"Division: {a} / {b} = {a / b}");
        else
            Console.WriteLine("Division: Error! Division by zero is
not allowed.");
    }

    static void Main(string[] args)
    {
        Console.WriteLine("=== Practical 3(a): C# Delegates ===");

        // Step 3: Instantiate delegates and bind them to methods
        MathOperation addDel = new MathOperation(Add);
        MathOperation subDel = new MathOperation(Subtract);
        MathOperation mulDel = new MathOperation(Multiply);
        MathOperation divDel = new MathOperation(Divide);

        // Step 4: Call the methods using delegates
        addDel(20, 10);
        subDel(20, 10);
        mulDel(20, 10);
        divDel(20, 10);
    }
```

```
    }
}
```

Output:



b. Write a program using multicast delegate.

Code:

```
using System;

namespace Practical3B
{
    // Step 1: Declare a delegate
    public delegate void Notify(string message);

    class Program
    {
        public static void DisplayOnConsole(string message)
        {
            Console.WriteLine($"Console Notification: {message}");
        }

        public static void DisplayInUppercase(string message)
        {
            Console.WriteLine($"Uppercase Notification:
{message.ToUpper()}");
        }

        static void Main(string[] args)
        {
            Console.WriteLine("=== Practical 3(b): Multicast Delegate
===");
```

```
            // Step 2: Create delegate instances
            Notify notify1 = DisplayOnConsole;
            Notify notify2 = DisplayInUppercase;

            // Step 3: Combine delegates using '+' operator to create a
multicast delegate
            Notify multiDelegate = notify1 + notify2;

            // Step 4: Invoke the multicast delegate (it will execute both
methods sequentially)
            multiDelegate("Welcome to C# Delegates Practical!");
        }
    }
}
```

Output:

```
=== Practical 3(b): Multicast Delegate ===
Console Notification: Welcome to C# Delegates Practical!
Uppercase Notification: WELCOME TO C# DELEGATES PRACTICAL!
```

c. **Create a class BankAccount with AccountNumber and Balance. Implement property for Balance with validation (Balance cannot be negative).**

Code:

```
using System;

namespace Practical3C
{
    class BankAccount
    {
        public string AccountNumber { get; set; }
```

```
        private decimal balance;

        // Property with validation (Balance cannot be negative)
        public decimal Balance
        {
            get
            {
                return balance;
            }
            set
            {
                if (value < 0)
                {
                    Console.WriteLine("Validation Error: Balance cannot
be negative! Assignment rejected.");
                }
                else
                {
                    balance = value;
                    Console.WriteLine("Success: Balance updated
successfully.");
                }
            }
        }
    }

     class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== Practical 3(c): BankAccount
Validation ===");

            BankAccount acc = new BankAccount();
            acc.AccountNumber = "ACC-987654";
```

```
        Console.WriteLine($"Account Number:
{acc.AccountNumber}\n");

        // Testing with a valid balance
        Console.WriteLine("Attempting to set balance to 15000:");
        acc.Balance = 15000;
        Console.WriteLine($"Current Balance: {acc.Balance}\n");

        // Testing with a negative balance (Validation check)
        Console.WriteLine("Attempting to set balance to -5000:");
        acc.Balance = -5000;
        Console.WriteLine($"Current Balance remains:
{acc.Balance}");
    }
  }
}
```

Output:



Practical No: 4

Create an ASP.NET Web Forms application using simple server controls.
a. Design a Student Registration Form using the following ASP.NET controls:
i. Label and TextBox to accept Student Name
ii. RadioButton to select Gender (Male/Female)
iii. DropDownList to select Course (BSc IT, BSc CS, BCA)

iv. CheckBox for accepting Terms & Conditions
v. Calendar control to select Date
vi. Button control to submit the form vii. Label control to display the output

Code:

Student Registration.aspx

```
<%@ Page Language="C#" AutoEventWireup="true"
CodeBehind="StudentRegistration.aspx.cs"
Inherits="Practical4.StudentRegistration" %>

<!DOCTYPE html>
<html>
<head runat="server">
   <title>Student Registration Form</title>
</head>
<body>
   <form id="form1" runat="server">
      <div style="margin: 20px;">
         <h2>Student Registration Form</h2>

         <!-- i. Student Name -->
         <asp:Label ID="lblName" runat="server" Text="Student
Name: "></asp:Label>
         <asp:TextBox ID="txtStudentName"
runat="server"></asp:TextBox><br /><br />

         <!-- ii. Gender -->
         <asp:Label ID="lblGender" runat="server" Text="Gender:
"></asp:Label>
         <asp:RadioButton ID="rbMale" runat="server"
GroupName="Gender" Text="Male" />
         <asp:RadioButton ID="rbFemale" runat="server"
GroupName="Gender" Text="Female" /><br /><br />
```

```
        <!-- iii. Course DropDownList -->
        <asp:Label ID="lblCourse" runat="server" Text="Select
Course: "></asp:Label>
        <asp:DropDownList ID="ddlCourse" runat="server">
           <asp:ListItem Text="--Select Course--" Value="" />
           <asp:ListItem Text="BSc IT" Value="BSc IT" />
           <asp:ListItem Text="BSc CS" Value="BSc CS" />
           <asp:ListItem Text="BCA" Value="BCA" />
        </asp:DropDownList><br /><br />

        <!-- iv. Terms & Conditions CheckBox -->
        <asp:CheckBox ID="chkTerms" runat="server" Text="I
accept Terms & Conditions" /><br /><br />

        <!-- v. Calendar Control -->
        <asp:Label ID="lblDate" runat="server" Text="Select Date
of Birth: "></asp:Label><br />
        <asp:Calendar ID="CalendarDOB"
runat="server"></asp:Calendar><br /><br />

        <!-- vi. Submit Button -->
        <asp:Button ID="btnSubmit" runat="server"
Text="Submit" OnClick="btnSubmit_Click" /><br /><br />

        <!-- vii. Output Label -->
        <asp:Label ID="lblOutput" runat="server"
ForeColor="Green" Font-Bold="true"></asp:Label>
      </div>
   </form>
</body>
</html>
```

Student Registration.aspx.cs

```
using System;

namespace Practical4
```

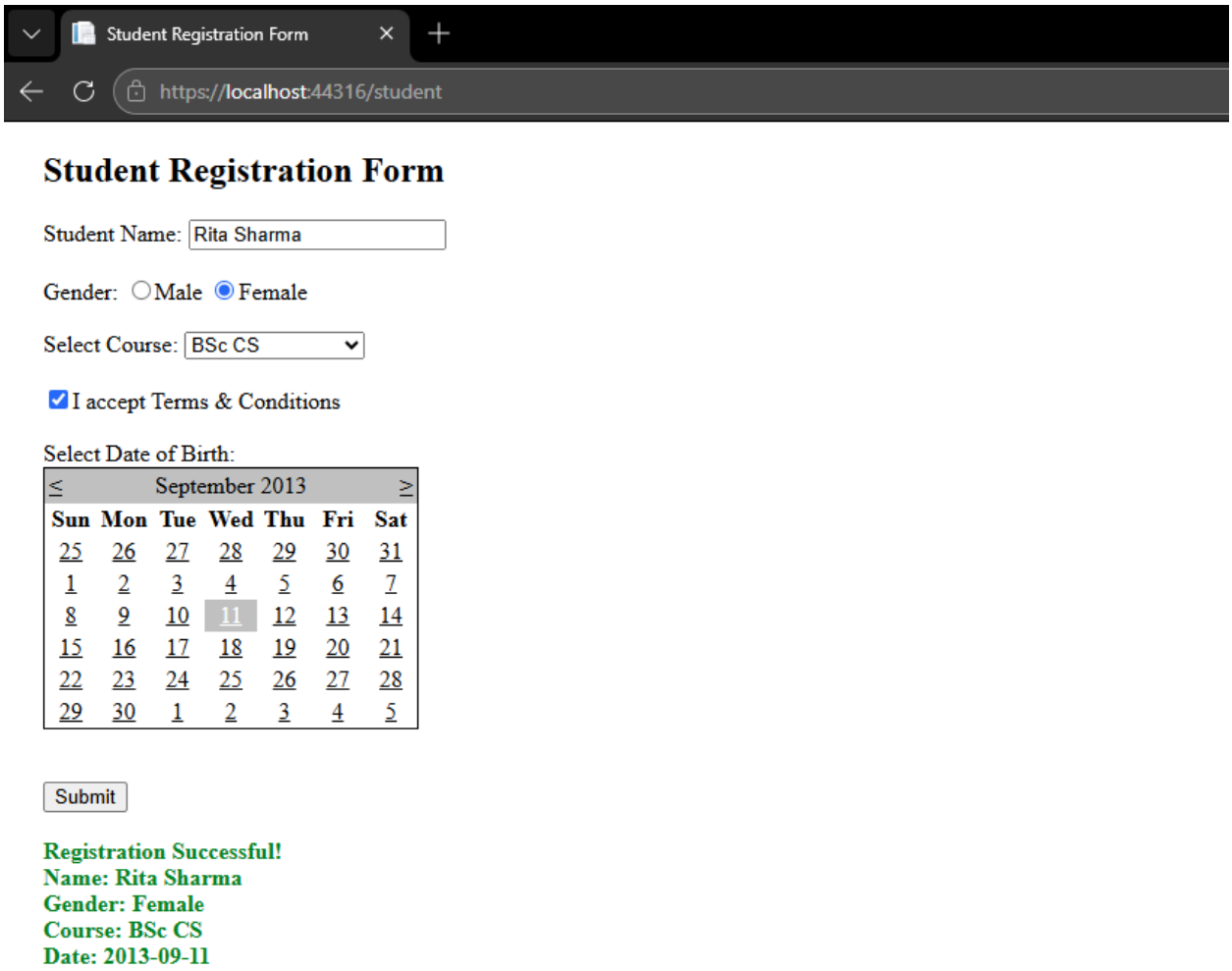
```
{
    public partial class StudentRegistration : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
        }

        protected void btnSubmit_Click(object sender, EventArgs e)
        {
            if (!chkTerms.Checked)
            {
                lblOutput.ForeColor = System.Drawing.Color.Red;
                lblOutput.Text = "You must accept Terms & Conditions!";
                return;
            }

            string gender = rbMale.Checked ? "Male" :
(rbFemale.Checked ? "Female" : "Not Specified");
            string selectedDate =
CalendarDOB.SelectedDate.ToString("yyyy-MM-dd");
            if (CalendarDOB.SelectedDate == DateTime.MinValue)
            {
                selectedDate = DateTime.Now.ToString("yyyy-MM-dd");
            }

            lblOutput.ForeColor = System.Drawing.Color.Green;
            lblOutput.Text = $"Registration Successful!<br />" +
                             $"Name: {txtStudentName.Text}<br />" +
                             $"Gender: {gender}<br />" +
                             $"Course: {ddlCourse.SelectedValue}<br />" +
                             $"Date: {selectedDate}";
        }
    }
}
```

Output:



b. Create a Registration form to demonstrate use of various Validation controls.

Code:

Validation.aspx

<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="ValidationForm.aspx.cs" Inherits="WebApplication17.ValidationForm" %>

<!DOCTYPE html>

```
<html>
<head runat="server">
   <title>Registration with Validations</title>
</head>
<body>
   <form id="form1" runat="server">
      <div style="margin: 20px;">
         <h2>Registration Form with Validations</h2>

         <!-- Name Validation -->
         <asp:Label ID="lblName" runat="server" Text="Name:
"></asp:Label>
         <asp:TextBox ID="txtName" runat="server"></asp:TextBox>
         <asp:RequiredFieldValidator ID="rfvName" runat="server"
ControlToValidate="txtName"
            ErrorMessage="Name is required"
ForeColor="Red">*</asp:RequiredFieldValidator><br /><br />

         <!-- Age Validation (RangeValidator) -->
         <asp:Label ID="lblAge" runat="server" Text="Age (18-60):
"></asp:Label>
         <asp:TextBox ID="txtAge" runat="server"></asp:TextBox>
         <asp:RequiredFieldValidator ID="rfvAge" runat="server"
ControlToValidate="txtAge"
            ErrorMessage="Age is required"
ForeColor="Red">*</asp:RequiredFieldValidator>
         <asp:RangeValidator ID="rvAge" runat="server"
ControlToValidate="txtAge"
            MinimumValue="18" MaximumValue="60" Type="Integer"
            ErrorMessage="Age must be between 18 and 60"
ForeColor="Red">Age must be 18-60</asp:RangeValidator><br /><br
/>

         <!-- Email Validation (RegularExpressionValidator) -->
         <asp:Label ID="lblEmail" runat="server" Text="Email:
"></asp:Label>
         <asp:TextBox ID="txtEmail" runat="server"></asp:TextBox>
```

```
        <asp:RegularExpressionValidator ID="revEmail"
runat="server" ControlToValidate="txtEmail"
            ValidationExpression="^[\w-\.]+@([\w-]+\.)+[\w-]{2,4}$"
            ErrorMessage="Invalid Email Format"
ForeColor="Red">Invalid
Email</asp:RegularExpressionValidator><br /><br />

        <!-- Password & CompareValidator -->
        <asp:Label ID="lblPass" runat="server" Text="Password:
"></asp:Label>
        <asp:TextBox ID="txtPass" runat="server"
TextMode="Password"></asp:TextBox><br /><br />

        <asp:Label ID="lblConfirmPass" runat="server"
Text="Confirm Password: "></asp:Label>
        <asp:TextBox ID="txtConfirmPass" runat="server"
TextMode="Password"></asp:TextBox>
        <asp:CompareValidator ID="cvPass" runat="server"
ControlToValidate="txtConfirmPass"
            ControlToCompare="txtPass" ErrorMessage="Passwords do
not match" ForeColor="Red">Passwords must
match</asp:CompareValidator><br /><br />

        <asp:Button ID="btnSubmit" runat="server" Text="Register"
OnClick="btnSubmit_Click" /><br /><br />

        <asp:ValidationSummary ID="ValSummary" runat="server"
ForeColor="Red" HeaderText="Please fix the following errors:" />
        <asp:Label ID="lblMsg" runat="server" ForeColor="Green"
Font-Bold="true"></asp:Label>
      </div>
   </form>
</body>
</html>
```

Validation.aspx.cs

```
<%@ Page Language="C#" AutoEventWireup="true"
CodeBehind="ValidationForm.aspx.cs"
Inherits="WebApplication17.ValidationForm" %>

<!DOCTYPE html>
<html>
<head runat="server">
   <title>Registration with Validations</title>
</head>
<body>
   <form id="form1" runat="server">
      <div style="margin: 20px;">
         <h2>Registration Form with Validations</h2>

         <!-- Name Validation -->
         <asp:Label ID="lblName" runat="server" Text="Name:
"></asp:Label>
         <asp:TextBox ID="txtName" runat="server"></asp:TextBox>
         <asp:RequiredFieldValidator ID="rfvName" runat="server"
ControlToValidate="txtName"
            ErrorMessage="Name is required"
ForeColor="Red">*</asp:RequiredFieldValidator><br /><br />

         <!-- Age Validation (RangeValidator) -->
         <asp:Label ID="lblAge" runat="server" Text="Age (18-60):
"></asp:Label>
         <asp:TextBox ID="txtAge" runat="server"></asp:TextBox>
         <asp:RequiredFieldValidator ID="rfvAge" runat="server"
ControlToValidate="txtAge"
            ErrorMessage="Age is required"
ForeColor="Red">*</asp:RequiredFieldValidator>
         <asp:RangeValidator ID="rvAge" runat="server"
ControlToValidate="txtAge"
            MinimumValue="18" MaximumValue="60" Type="Integer"
            ErrorMessage="Age must be between 18 and 60"
ForeColor="Red">Age must be 18-60</asp:RangeValidator><br /><br
/>
```

```
        <!-- Email Validation (RegularExpressionValidator) -->
        <asp:Label ID="lblEmail" runat="server" Text="Email:
"></asp:Label>
        <asp:TextBox ID="txtEmail" runat="server"></asp:TextBox>
        <asp:RegularExpressionValidator ID="revEmail"
runat="server" ControlToValidate="txtEmail"
            ValidationExpression="^[\w-\.]+@([\w-]+\.)+[\w-]{2,4}$"
            ErrorMessage="Invalid Email Format"
ForeColor="Red">Invalid
Email</asp:RegularExpressionValidator><br /><br />

        <!-- Password & CompareValidator -->
        <asp:Label ID="lblPass" runat="server" Text="Password:
"></asp:Label>
        <asp:TextBox ID="txtPass" runat="server"
TextMode="Password"></asp:TextBox><br /><br />

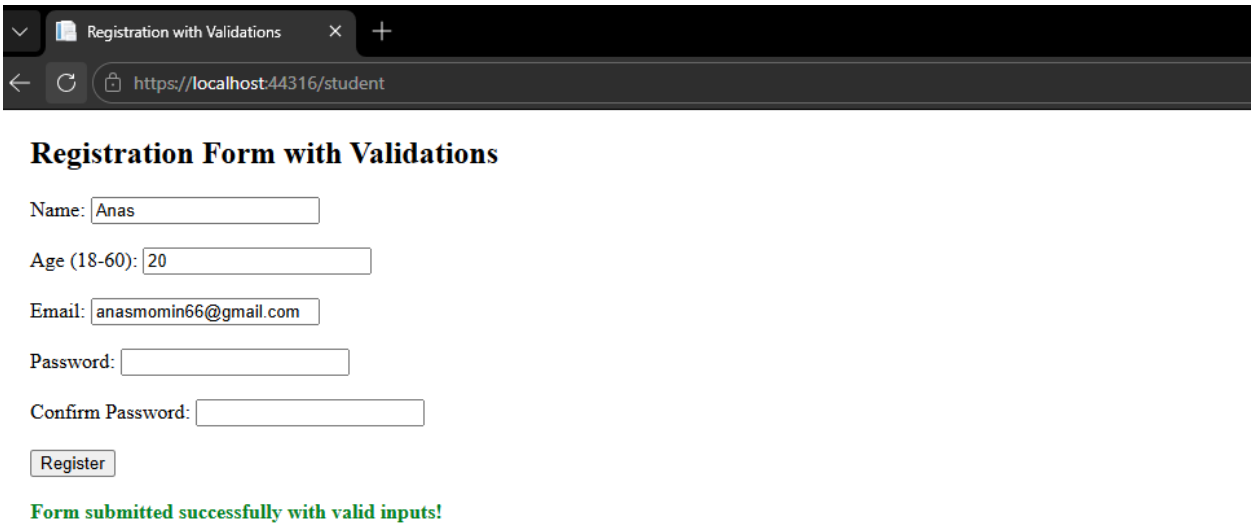
        <asp:Label ID="lblConfirmPass" runat="server"
Text="Confirm Password: "></asp:Label>
        <asp:TextBox ID="txtConfirmPass" runat="server"
TextMode="Password"></asp:TextBox>
        <asp:CompareValidator ID="cvPass" runat="server"
ControlToValidate="txtConfirmPass"
            ControlToCompare="txtPass" ErrorMessage="Passwords do
not match" ForeColor="Red">Passwords must
match</asp:CompareValidator><br /><br />

        <asp:Button ID="btnSubmit" runat="server" Text="Register"
OnClick="btnSubmit_Click" /><br /><br />

        <asp:ValidationSummary ID="ValSummary" runat="server"
ForeColor="Red" HeaderText="Please fix the following errors:" />
        <asp:Label ID="lblMsg" runat="server" ForeColor="Green"
Font-Bold="true"></asp:Label>
      </div>
   </form>
```

```
</body>
</html>
```

Output:



c. Create Web Form to demonstrate use of Adrotator Control.

Code:

XML File:

```
<?xml version="1.0" encoding="utf-8" ?>
<Advertisements>
   <Ad>

<ImageUrl>https://www.google.com/images/branding/googlelogo/1x/goo
glelogo_color_272x92dp.png</ImageUrl>
      <NavigateUrl>https://www.google.com</NavigateUrl>
      <AlternateText>Google Ad</AlternateText>
      <Impressions>10</Impressions>
```

```
    <Keyword>Search</Keyword>
  </Ad>
  <Ad>
    <ImageUrl>https://www.microsoft.com/favicon.ico</ImageUrl>
    <NavigateUrl>https://www.microsoft.com</NavigateUrl>
    <AlternateText>Microsoft Ad</AlternateText>
    <Impressions>20</Impressions>
    <Keyword>Tech</Keyword>
  </Ad>
</Advertisements>
```

AdRotator.aspx

```
<%@ Page Language="C#" AutoEventWireup="true"
CodeBehind="AdRotatorDemo.aspx.cs"
Inherits="WebApplication17.AdRotatorDemo" %>

<!DOCTYPE html>
<html>
<head runat="server">
  <title>AdRotator Control Demo</title>
</head>
<body>
  <form id="form1" runat="server">
    <div style="margin: 20px; text-align: center;">
      <h2>AdRotator Control Example</h2>
      <br />
      <asp:AdRotator ID="AdRotator1" runat="server"
AdvertisementFile="~/Ads.xml" />
    </div>
  </form>
</body>
</html>
```

Practical No: 5

a. **Create a web application to demonstrate the use of different types of Cookies.**

Code:

```
using System;
using System.Web;

namespace WebApplication17
{
   public partial class CookieDemo : System.Web.UI.Page
   {
      protected void Page_Load(object sender, EventArgs e)
      {
         // 1. Setting a Non-Persistent (Session) Cookie
         HttpCookie sessionCookie = new HttpCookie("UserSession",
"JohnDoe");
         Response.Cookies.Add(sessionCookie);

         // 2. Setting a Persistent Cookie (Expires in 7 days)
         HttpCookie persistentCookie = new HttpCookie("UserRole",
"Administrator");
         persistentCookie.Expires = DateTime.Now.AddDays(7);
         Response.Cookies.Add(persistentCookie);

         // 3. Reading the Cookies
         if (Request.Cookies["UserSession"] != null)
         {
            string sessionVal = Request.Cookies["UserSession"].Value;
            Response.Write("Session Cookie Value: " + sessionVal +
"<br/>");
         }

         if (Request.Cookies["UserRole"] != null)
         {
            string persistentVal = Request.Cookies["UserRole"].Value;
```

```
            Response.Write("Persistent Cookie Value: " +
persistentVal + "<br/>");
        }
      }
   }
}
```

b. Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core.

Code:

Step 1: Product.cs

```
public class Product
{
   public int Id { get; set; }
   public string Name { get; set; }
   public decimal Price { get; set; }
   public int Stock { get; set; }
}
```

Step 2: AppDbContext.cs

```
using System.Data.Entity;

namespace WebApplication18
{
    public class AppDbContext : DbContext
    {

        public AppDbContext() :
base(@"Server=(localdb)\MSSQLLocalDB;Database=ProductDB;Tr
usted_Connection=True;")
{
```

```
}

 public DbSet<Product> Products { get; set; }
    }
}
```

ProductInsert.aspx

```
<%@ Page Language="C#" AutoEventWireup="true"
CodeBehind="ProductInsert.aspx.cs"
Inherits="WebApplication18.ProductInsert" %>

<!DOCTYPE html>
<html>
<head runat="server">
   <title>Insert Product using EF</title>
</head>
<body>
   <form id="form1" runat="server">
      <div style="margin: 30px;">
         <h2>Add Product to Database</h2>

         <asp:Label ID="lblName" runat="server" Text="Product
Name: "></asp:Label>
         <asp:TextBox ID="txtName"
runat="server"></asp:TextBox><br /><br />

         <asp:Label ID="lblPrice" runat="server" Text="Price:
"></asp:Label>
         <asp:TextBox ID="txtPrice"
runat="server"></asp:TextBox><br /><br />

         <asp:Label ID="lblStock" runat="server" Text="Stock:
"></asp:Label>
         <asp:TextBox ID="txtStock"
runat="server"></asp:TextBox><br /><br />
```

```
        <asp:Button ID="btnInsert" runat="server" Text="Insert
Product" OnClick="btnInsert_Click" /><br /><br />

        <asp:Label ID="lblMsg" runat="server"
ForeColor="Green" Font-Bold="true"></asp:Label>
    </div>
  </form>
</body>
</html>
```

ProductInsert.aspx.cs

```
using System;

namespace WebApplication18
{
  public partial class ProductInsert : System.Web.UI.Page
  {
    protected void Page_Load(object sender, EventArgs e)
    {
    }

    protected void btnInsert_Click(object sender, EventArgs e)
    {
      using (var context = new AppDbContext())
      {
        var prod = new Product
        {
          Name = txtName.Text,
          Price = Convert.ToDecimal(txtPrice.Text),
          Stock = Convert.ToInt32(txtStock.Text)
        };

        context.Products.Add(prod);
        context.SaveChanges();
```

```
            lblMsg.Text = "Product Inserted Successfully into
Database!";
        }
      }
   }
}
```

c. **Develop a web application to perform CRUD Operations (Create, Read, Update, Delete) on Employee table using EF Core.**

EmployeeCrud.aspx

```
<asp:GridView ID="gvEmployees" runat="server"
AutoGenerateColumns="False"
   OnRowDeleting="gvEmployees_RowDeleting"
OnRowEditing="gvEmployees_RowEditing"
   OnRowUpdating="gvEmployees_RowUpdating"
OnRowCancelingEdit="gvEmployees_RowCancelingEdit">

   <Columns>
      <asp:BoundField DataField="Id" HeaderText="ID"
ReadOnly="True" />
      <asp:BoundField DataField="Name" HeaderText="Name" />
      <asp:BoundField DataField="Department"
HeaderText="Dept" />
      <asp:CommandField ShowEditButton="True"
ShowDeleteButton="True" />
   </Columns>
</asp:GridView>
<br />
Name: <asp:TextBox ID="txtName"
runat="server"></asp:TextBox>
Dept: <asp:TextBox ID="txtDept" runat="server"></asp:TextBox>
<asp:Button ID="btnAdd" runat="server" Text="Add Employee"
OnClick="btnAdd_Click" />
```

Employee.cs

```
namespace WebApplication18
{
   public class Employee
   {
      public int Id { get; set; }
      public string Name { get; set; }
      public string Department { get; set; }
   }
}
```

EmployeeDbContext.cs

```
using System.Data.Entity;

namespace WebApplication18
{
   public class EmployeeDbContext : DbContext
   {
      public EmployeeDbContext() :
base(@"Server=(localdb)\MSSQLLocalDB;Database=EmployeeDB;
Trusted_Connection=True;")
      {
      }

      public DbSet<Employee> Employees { get; set; }
   }
}
```

EmployeeCrud.aspx

```
<%@ Page Language="C#" AutoEventWireup="true"
CodeBehind="EmployeeCrud.aspx.cs"
Inherits="WebApplication18.EmployeeCrud" %>

<!DOCTYPE html>
<html>
<head runat="server">
   <title>Employee CRUD</title>
</head>
<body>
   <form id="form1" runat="server">
      <div style="margin: 20px;">
         <h2>Employee Management (CRUD)</h2>

         <asp:GridView ID="gvEmployees" runat="server"
AutoGenerateColumns="False"
            OnRowDeleting="gvEmployees_RowDeleting"
OnRowEditing="gvEmployees_RowEditing"
            OnRowUpdating="gvEmployees_RowUpdating"
OnRowCancelingEdit="gvEmployees_RowCancelingEdit">
            <Columns>
               <asp:BoundField DataField="Id" HeaderText="ID"
ReadOnly="True" />
               <asp:BoundField DataField="Name"
HeaderText="Name" />
               <asp:BoundField DataField="Department"
HeaderText="Dept" />
               <asp:CommandField ShowEditButton="True"
ShowDeleteButton="True" />
            </Columns>
         </asp:GridView>
         <br />
         Name: <asp:TextBox ID="txtName"
runat="server"></asp:TextBox>
         Dept: <asp:TextBox ID="txtDept"
runat="server"></asp:TextBox>
```

```
        <asp:Button ID="btnAdd" runat="server" Text="Add
Employee" OnClick="btnAdd_Click" />
      </div>
   </form>
</body>
</html>
```

d. Authentication and Authorization in ASP.NET Core.

Web.config

```
<configuration>
  <system.web>
    <authentication mode="Forms">
      <forms loginUrl="Login.aspx"
defaultUrl="Dashboard.aspx" timeout="30" />
    </authentication>

    <authorization>
      <deny users="?" />
    </authorization>
  </system.web>
</configuration>
```

Login.aspx

```
<%@ Page Language="C#" AutoEventWireup="true"
CodeBehind="Login.aspx.cs" Inherits="WebApplication18.Login"
%>

<!DOCTYPE html>
<html>
<head runat="server">
   <title>Login Page</title>
```

```
</head>
<body>
   <form id="form1" runat="server">
      <div style="margin: 50px;">
         <h2>User Login (Authentication)</h2>
         Username: <asp:TextBox ID="txtUser"
runat="server"></asp:TextBox><br /><br />
         Password: <asp:TextBox ID="txtPass" runat="server"
TextMode="Password"></asp:TextBox><br /><br />
         <asp:Button ID="btnLogin" runat="server" Text="Login"
OnClick="btnLogin_Click" /><br /><br />
         <asp:Label ID="lblError" runat="server"
ForeColor="Red"></asp:Label>
      </div>
   </form>
</body>
</html>
```

Login.aspx.cs

```
using System;
using System.Web.Security;

namespace WebApplication18
{
   public partial class Login : System.Web.UI.Page
   {
      protected void Page_Load(object sender, EventArgs e)
      {
      }

      protected void btnLogin_Click(object sender, EventArgs e)
      {

         if (txtUser.Text == "admin" && txtPass.Text == "12345")
         {
```

```
FormsAuthentication.RedirectFromLoginPage(txtUser.Text, false);
            }
            else
            {
                lblError.Text = "Invalid Username or Password!";
            }
        }
    }
}
```

Dashboard.aspx

```
<%@ Page Language="C#" AutoEventWireup="true"
CodeBehind="Dashboard.aspx.cs"
Inherits="WebApplication18.Dashboard" %>

<!DOCTYPE html>
<html>
<head runat="server">
    <title>Dashboard</title>
</head>
<body>
    <form id="form1" runat="server">
        <div style="margin: 50px;">
            <h2>Welcome to Dashboard! (Authorized Area)</h2>
            <p>Successfully Authenticate and Authorized </p>
            <asp:Button ID="btnLogout" runat="server"
Text="Logout" OnClick="btnLogout_Click" />
        </div>
    </form>
</body>
</html>
```

Dashboard.aspx.cs

```
using System;
```

```
using System.Web.Security;

namespace WebApplication18
{
    public partial class Dashboard : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
        }

        protected void btnLogout_Click(object sender, EventArgs e)

{
            FormsAuthentication.SignOut();
            Response.Redirect("Login.aspx");
        }
    }
}
```